



MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Santosh, Tangail-1902

LAB REPORT

Department of : *Information and Communication Technology*

Lab Report No : *01*

Lab Report On : *Data Analysis in Wireshark*

Course Title : *Computer Networks*

Course Code : *ICT3202*

Submitted by,

Name : *Kuldip Saha Mugdha*

ID : *IT32018* Session : *2021-22*

Year : *3rd* Semester : *2nd*

Dept. of *ICT* MBSTU

Submitted to,

Mn. Dr. Nazrul Islam

Associate professor

Dept. of ICT

MBSTU

Date of Performance : *21/10/25*

Date of Submission : *28/10/25*

Introduction:

Purpose of the experiment:

The purpose of this experiment was to analyze network communication by capturing and examining data packets using Wireshark. The goal was to understand how different network protocols interact and to visualize packet flow in real time.

Background Information:

Wireshark is an open-source network protocol analyzer that captures and displays packets in details. It helps study protocols like Ethernet, IP, TCP, UDP, ICMP, DNS and HTTP. By decoding packets, users can observe/analyze protocols headers, encapsulation, and payload data. Wireshark is commonly used for network troubleshooting, security auditing, and teaching network concepts.

Hypothesis:

It was expected that Wireshark would successfully capture and display various protocol packets, allowing clear analysis of how data travels between source and destination across layers.

Objective:

- To install and configure Wireshark for packet capture.
- To capture and analyze live network traffic.
- To identify and interpret protocol fields and their functions.
- To use filters for isolating specific protocols and studying them in details.

Materials and Methods (Procedure):

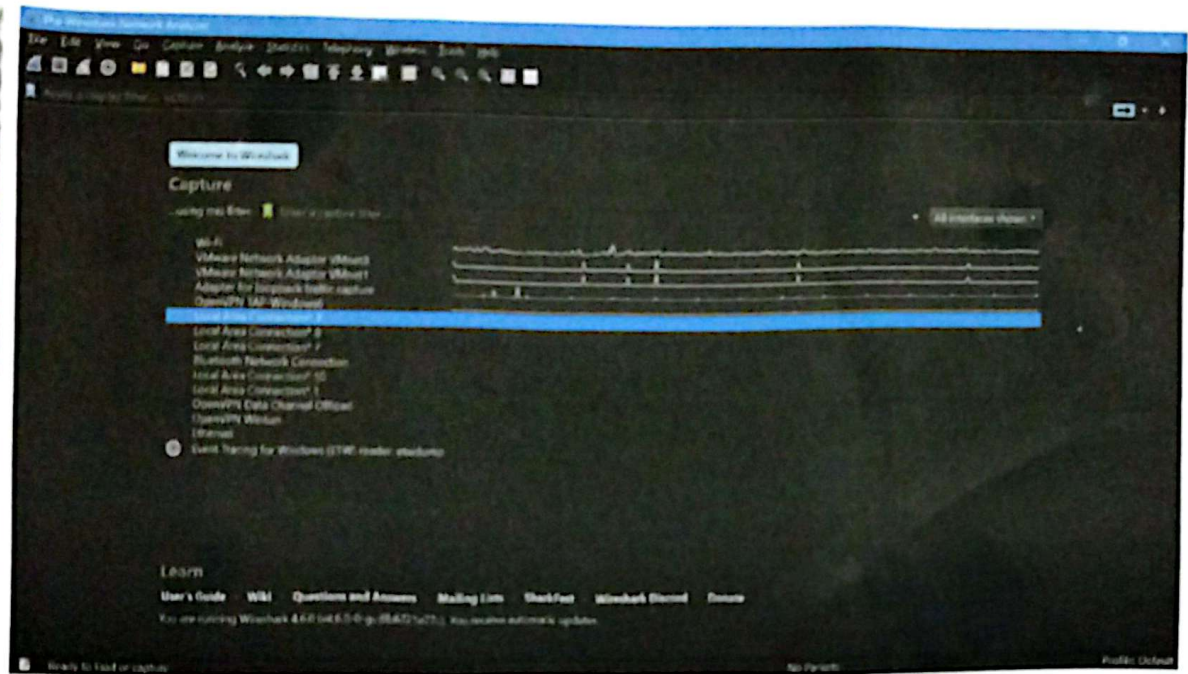
List of Materials:

- Computer or laptop with active internet connection.
- Software: Wireshark (v4.6.0)
- Libraries: WinPcap (for packet capture)
- Operating System: Windows 11.
- Network Interface Card (NIC).

Procedure:

1. Wireshark was downloaded from <https://www.wireshark.org> and installed with WinPcap.
2. The program was launched, and then active network interface was selected.

3. Packet capture was started by clicking the start capturing packets.



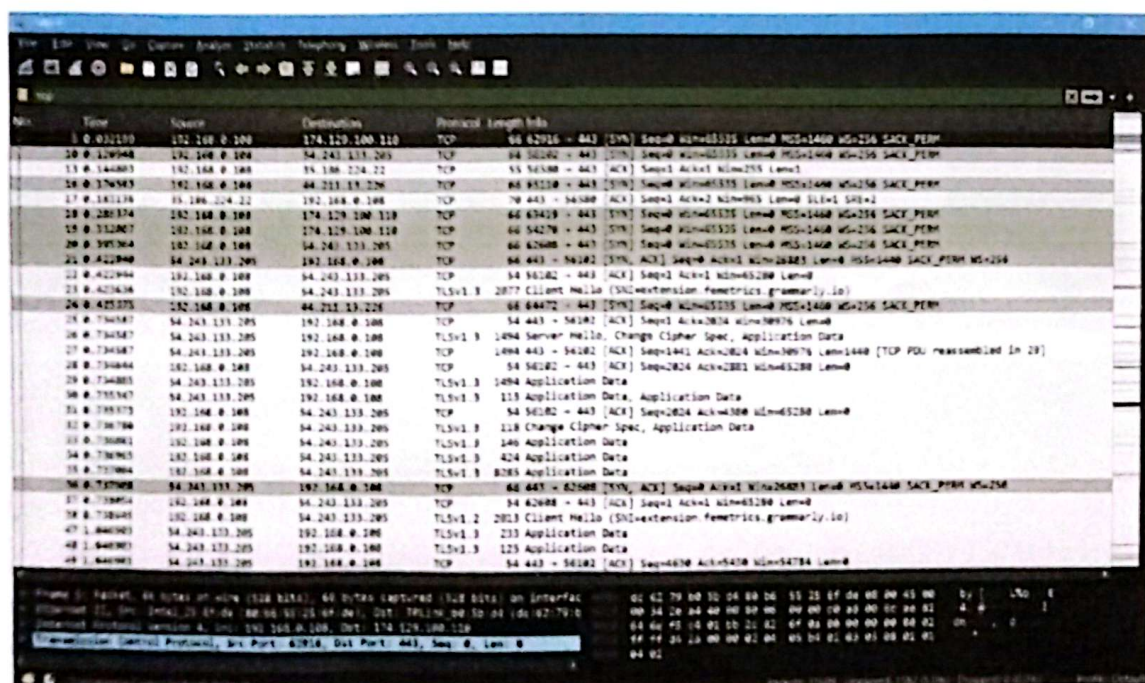
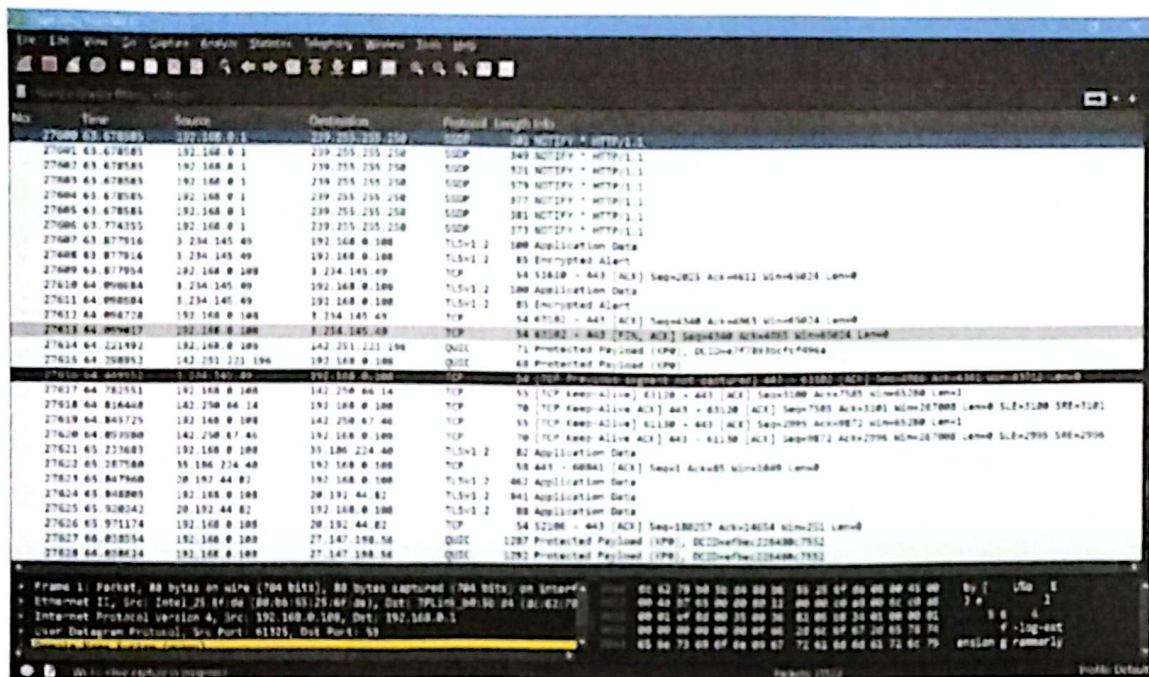
4. Network activity was generated to capture real traffic.

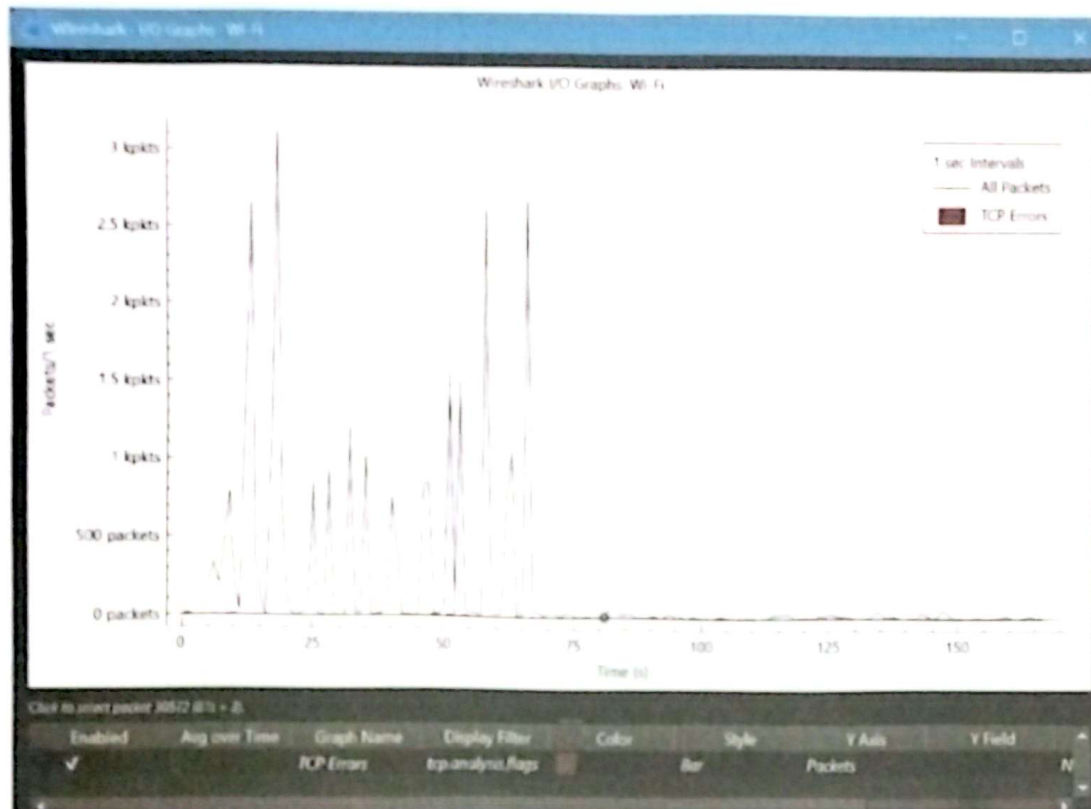
5. The capture was stopped after sufficient data collection.

6. Display filters (e.g., TCP) were applied to analyze specific protocol.

7. Protocol details such as source/destination IP, MAC addresses, and TCP flags were studied.

8. Flow graph feature was used to visualize communication between hosts.





The screenshot shows the Windows Task Manager Performance tab. On the left, the 'Processes' list is expanded, showing 'System' at the top. The main pane displays a table of system processes. The table has the following columns: Name, Private, System, % Private, % System, No. Threads, and No. Bytes. The 'System' process is highlighted in blue.

Name	Private	System	% Private	% System	No. Threads	No. Bytes
System	2,100 bytes	0	0 bytes	0	2	100 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
csrss.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes
smss.exe	100 bytes	0	0 bytes	0	1	100 bytes
svchost.exe	1,000 bytes	0	0 bytes	0	1	1,000 bytes</

[illegible]

8. The captured data was saved in .pcap format for record and analysis.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Ethernet	100.0	27609	100.0	29341329	1210 k	0	0	0
Internet Protocol Version 4	100.0	27609	1.0	442526	20 k	0	0	0
User Datagram Protocol	0.1	16	0.0	640	30	0	0	0
Multicast Domain Name System	0.0	15	0.0	120	5	0	0	0
Internet Control Message Protocol v4	0.0	15	0.0	2375	109	15	2325	109
Internet Protocol Version 4	0.0	1	0.0	20	1	1	20	1
User Datagram Protocol	99.9	27601	2.2	61664	25 k	0	0	0
Single Service Discovery Protocol	99.9	27601	2.2	27064	11 k	0	0	0
OSPF	0.1	25	0.0	4872	227	25	4872	227
Multicast Domain Name System	0.0	14	0.0	220	105	14	220	105
Dynamic Host Configuration Protocol	0.0	2	0.0	400	20	2	400	20
Domain Name System	0.0	264	0.1	28076	952	264	28076	952
Data	0.0	5	0.0	364	12	5	364	12
Transmission Control Protocol	0.0	1502	0.1	34022	1000	1502	34022	1000
Transport Layer Security	1.0	616	2.0	34010	27 k	616	34010	27 k
Hypertext Transfer Protocol	0.0	12	0.0	3606	109	12	3606	109
Data	0.0	25	0.0	116	5	25	116	5
Internet Group Management Protocol	0.1	16	0.0	232	10	16	232	10
Address Resolution Protocol	0.0	12	0.0	136	15	12	136	15
IEEE 1Q Virtual LAN	0.0	1790	0.0	7180	236	0	0	0

Safety precautions:

→ Captures were performed only on the local system network to avoid privacy violations.

→ No unauthorized or encrypted user data was analyzed.

Results:

Data Presentation:

Protocol	Source IP	Destination IP	Description
ARP	192.168.1.5	192.168.1.1	MAC address resolution query.
ICMP	192.168.1.5	P.P.P.P	Echo Request (Ping)
TCP	192.168.1.5	192.250.39.39	SYN-TCP Connection Initiation.
HTTP	192.168.1.5	192.250.39.39	Get Request for web data.

Observation:

- Packets were captured continuously while browsing.
- Different protocols layers were clearly visible (Ethernet → IP → TCP → HTTP).
- The TCP three-way handshake (SYN, SYN-Ack, ACK) was observed.
- Filtering helped isolate traffic efficiently.
- Flow graphs displayed sequentially to build a communication between client and server.

Discussion:

Interpretation of Results:

The captured packets verified that data travels through multiple protocol layers, each adding its header for correct delivery.

Comparison with Theoretical Expectations:

The experiment matched expectations - packets were captured and analyzed successfully, and all common protocols behaved as described in networking theory.

Source of Error:

- Short capture duration might have missed some packets.
- Background applications could have introduced extra traffic.
- Network latency may have affected timing graphs.

Suggestions for Improvement:

- Use longer capture sessions for more complete analysis.
- Capture on a controlled local network to reduce background noise.
- Utilize advanced filters and coloring rules for protocol clarity.

Further Research:

Future studies could include analyzing encrypted HTTPS traffic, DNS resolution times, or network latency patterns under different loads.

Conclusion:

Summary of Findings:

Wireshark successfully captured and displayed network packets, showing protocol interactions and data exchange processes. The experiment provided a practical understanding of network layers, headers and data encapsulation.

Reflection on Hypothesis:

The hypothesis was supported - Wireshark effectively revealed how data is transmitted across the network and demonstrated protocol behavior in real time.

Practical Implications:

Wireshark is essential for network administrators and cybersecurity analysts to diagnose connectivity issues, optimize performance, and enhance network security through packet inspection.

References:

1/ Stenbenz, J.P.H. et al. (2018). Introduction to Protocol Analysis with Wireshark.

2/ Kurose, J.F., & Rom, K.W. (2017). Computer Networking: A Top-Down Approach (7th ed.)

3/ Wireshark Foundation (2029). {website}

Appendices:

Appendix A - Wireshark interface overview

Screenshot showing the packet list, details and Bytes panes.

Appendix B - TCP Handshake view

Captured SYN, SYN-ACK, and ACK packets.

Appendix C - Example Filters

- http → Display HTTP traffic
- icmp → Show ping packets.
- ip.addr == 192.168.1.1 → Filter traffic to/from a specific IP.



MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Santosh, Tangail-1902

LAB REPOT

Department of : Information and Communication Technology

Lab Report No : 02

Lab Report On : P2P Connection of two laptops using Ethernet cable

Course Title : Computer Network Lab

Course Code : ICT-3202

Submitted by

Name : Kuldeep Saha

ID : IT22018 Session : 2021-22

Year : 3rd Semester : 2nd

Dept. of ICT, MBSTU

Submitted to

Mn. Dr. Nazmul Islam
Associate Professor

Dept. of ICT, MBSTU

Date of Performance : 4/11/25

Date of Submission : 25/11/25

Introduction:

Purpose of the experiment: The purpose of this experiment is to create a direct network connection between two laptops using an ethernet cable and to enable file sharing between them without using a router or switch.

Background information: A computer network allows multiple devices to communicate, exchange information and share resources. When two laptops are connected with an ethernet cable, they form a peer-to-peer (P2P) network. By manually assigning static IP addresses, both devices can communicate directly according to networking principles (IP addressing, subnets, ARP).

Hypothesis: If both laptops are connected directly via ethernet and are configured with static IP addresses in the same subnets, then the devices will successfully communicate with each other and share files.

Objectives:

→ To establish a direct connection between two laptops using a ethernet cable.

→ To Configure static IP address for LAN communication.

→ To verify connectivity using the 'ping' command.

→ To share files between two laptops.

Materials and Methods:

Materials:

1) Two laptops with ethernet port.

2) One ethernet cable (cat5).

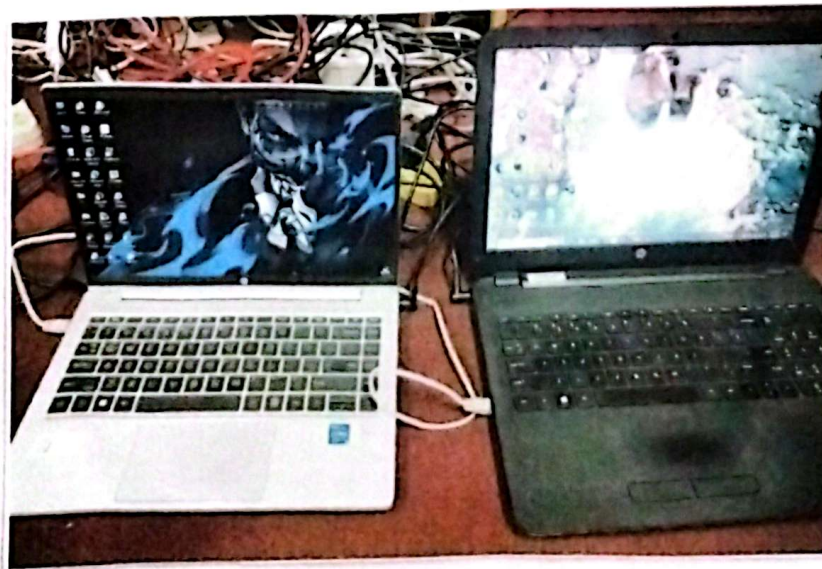
3) Windows Operating System on both laptops.

4) Administrator access.

5) File to share for testing.

Methods:

Step-1: physical connection:



Connect one ~~one~~ end of the ethernet cable to Laptop 1 and the other end to Laptop 2.

Step-2: Configure static IP Address:

1) Go to Control panel → Network and Sharing center → Change Adapter settings.

2) Right-click ethernet → Properties → IPv4 → Properties.

3) Set:

• Laptop 1:

→ IP: 192.168.0.1

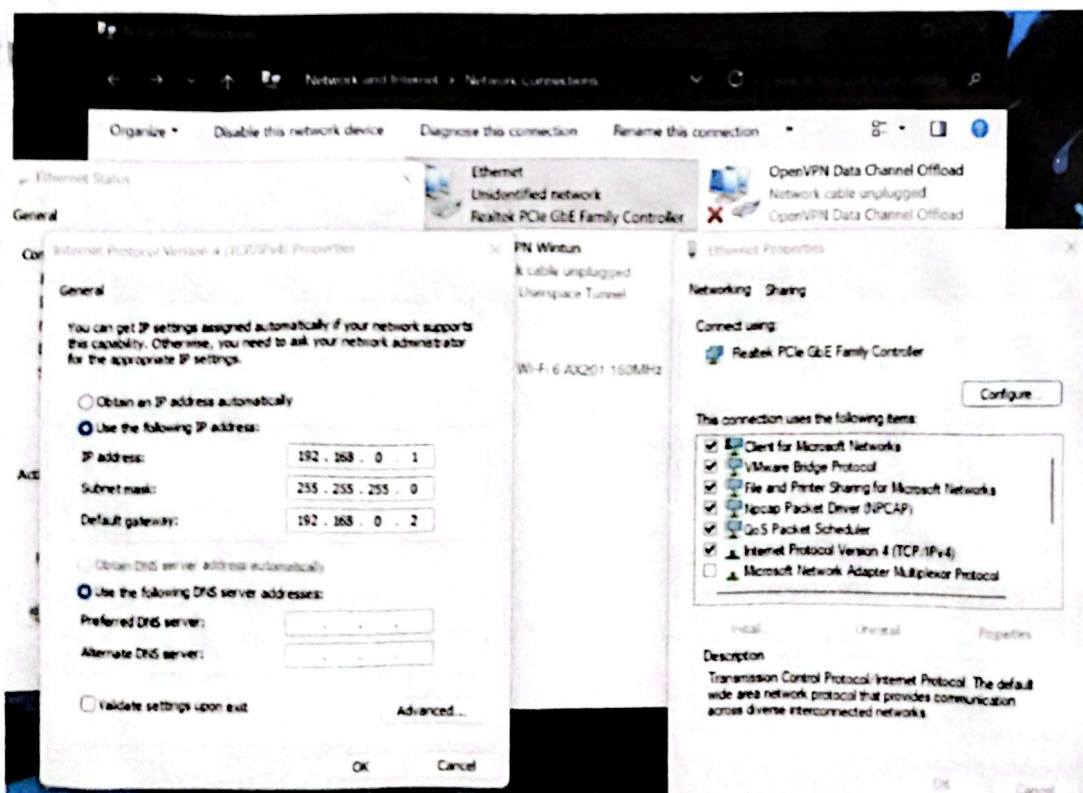
→ Subnet mask: 255.255.255.0

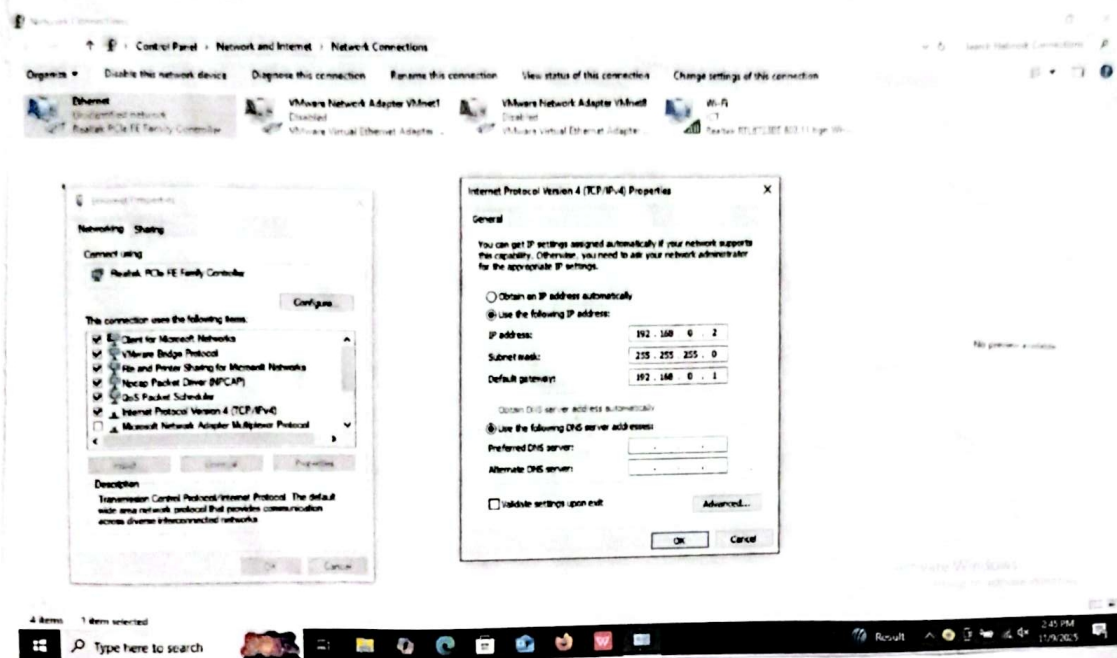
• Laptop 2:

→ IP: 192.168.0.2

→ Subnet mask: 255.255.255.0

4) Click OK on both laptops.





Step-3: Verify the IP Assignment:

Open Command prompt → Type: 'ipconfig'.

Confirm the correct IPv4 address.

Step-4: Test Connectivity using Ping:

From Laptop 1:

Ping 192.168.0.1

From Laptop 2:

Ping 192.168.0.2

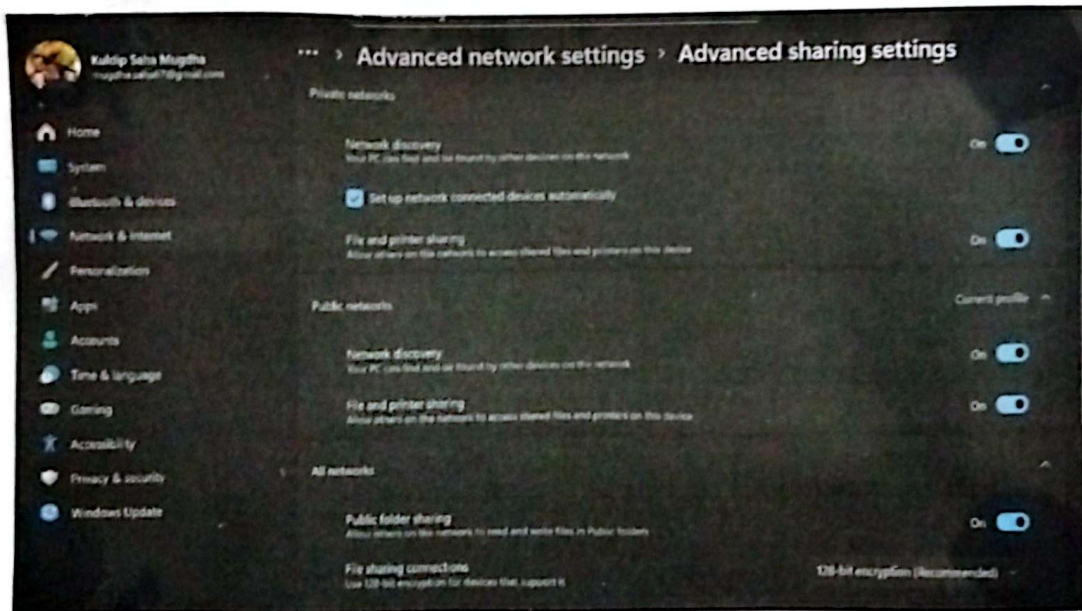
Step-5: Enable File sharing:

1) Open control panel → Network and sharing center → Advance sharing settings.

2) Enable:

- Network Discovery
- File and Printer sharing

3) Save changes.



Step-6 share a Folder:

1) Create a folder on Laptop 1 (Secret file)

Right click → Properties → sharing → Advance sharing

Enable "Share this folder".

From Laptop 2 ~~press~~ search in File Explorer

"\\MUGDHAG7\Secret file"

Choose people to share with

Type a name and then click Add, or click the arrow to find someone.

Name	Permission Level
Everyone	Read/Write
Kuldip Saha Mugdha (mugdha.saha67@gmail.com)	Owner

[I'm having trouble sharing](#)

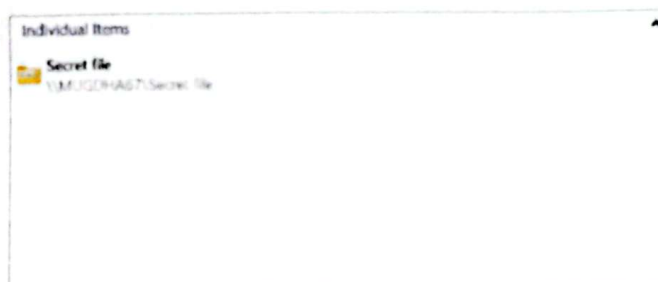
Share Cancel

Fig: Giving Access



Your folder is shared.

You can [send](#) someone links to these shared items, or [copy](#) and paste the links into another app.



[Show me all the network shares on this computer.](#)

Done

Fig: Sharing the file

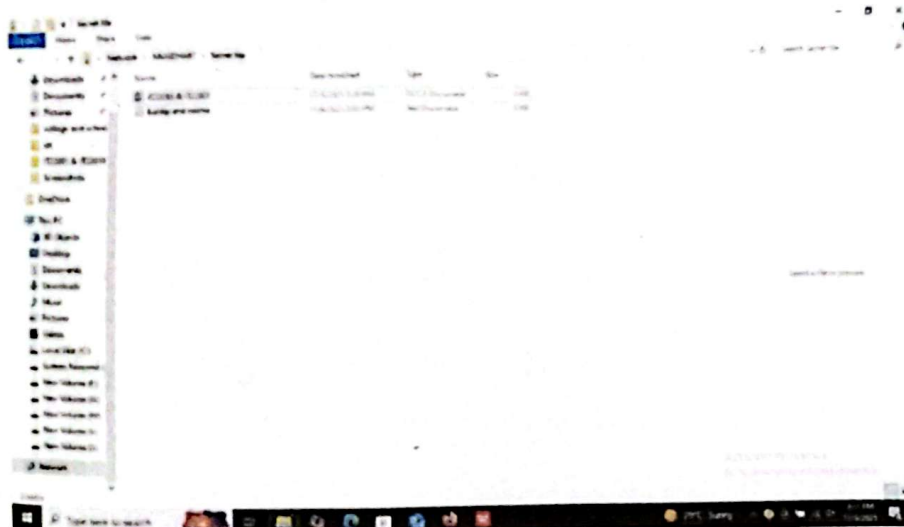


Fig: File shared in Laptop 2

Safety Precautions:

- Do not use damaged or loose Ethernet cables.
- Do not change important network configurations in public networks.
- Re-enable firewall after experiment.
- Handle laptops carefully while connecting cables.

Results:

Data presentation:

- Static IP configuration was successfully applied.
- Ping test succeeded, showing 0% packet loss.
- Shared folder on Laptop 1 was accessible on Laptop 2.
- Files were transferred without errors.

Observation:

- When IPs were not properly configured, ping returned "Request timed out".
- After setting both laptops to the same subnet, communication became successful.
- Disabling firewall temporarily was unnecessary for the ping test.

Discussion:

Interpretation of Results: Successful ping replies confirm that the two laptops established a functioning P2P network. Accessing the shared folder proves that file sharing worked through SMB protocol.

Comparison with theoretical expectations: Theory states that devices in the same subnet and connected via ethernet can communicate directly. The practical results matched this theory:

- Same subnet = communication possible
- Static IP = stable LAN connection.

Further Research:

- Creating LAN with more than two devices.
- Using switches/routers to form larger networks.
- Testing bandwidth and latency.
- Implementing network security in peer-to-peer networks.
- Exploring wireless file sharing method.

Conclusion:

Summary of Findings:

- Direct ethernet connection between two Laptops was successfully established.
- Static IP configuration enabled proper communication.
- Ping test verified active network connectivity.
- File sharing feature allowed successful file transfer.

Reflection of Hypothesis: The hypothesis was correct - assigning static IPs in the same subnet allowed the laptops to communicate and share files.

Practical Implications:

- Useful for fast file transfer without the internet.
- Helpful in computer labs, fieldwork or secure environments.
- Demonstrates core networking principles used in real-life LAN systems.

References:

- Microsoft Windows Networking Documentation.
- TCP/IP Model Concepts.
- CompTIA Network+ Study Guide.
- Networking Fundamentals - Cisco Academy.

Appendices:

Appendix A:

Full command used:

```
"ip config  
ping 192.168.0.X  
\\ 192.168.0.X
```

Appendix B:

Network Configuration Diagram

Laptop 1 (192.168.0.1) ← Ethernet Cable → Laptop 2 (192.168.0.2)

Appendix C:

All experiment ~~se~~ screenshots inserted in relevant section.



MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Santosh, Tangail-1902

LAB REPOT

Department of : Information and Communication Technology

Lab Report No : 3

Lab Report On : Basic Linux Command

Course Title : Computer Network Lab

Course Code : ICT-3202

Submitted by

Name : Kuldip Saha Mukherjee

ID : I722018 Session : 2021-22

Year : 3rd Semester : 2nd

Dept. of ICT, MBSTU

Submitted to

Mr. Dr. Nazrul Islam
Associate Professor

Dept. of ICT, MBSTU

Date of Performance : 18/11/25

Date of Submission : 2/12/25

Introduction:

Purpose of the experiment: The purpose of the experiment is to learn, practice and understand fundamental Linux Commands used for file management, directory navigation, permissions, networking and monitoring the system.

Background information: Linux is a widely used open source operating system known for its stability, security and command line based operations. Unlike windows, Linux relies heavily on the terminal for system control.

Hypothesis: If it is essential linux commands are executed correctly and understood conceptually, then the user will be able to navigate the operating system efficiently and perform tasks such as file manipulation, permission control, process management etc.

Objective:

→ To understand the purpose and usage of basic linux commands.

→ To learn file and directory handling through the terminal.

→ To change file permission and ownership using linux security concept.

→ To extract file system and network info using command line.

Materials and Methods:

Materials:

- i) A computer running Kali linux in vmware.
- ii) Terminal (Bash)
- iii) Internet connection,
- iv) A remote server account.

Method and procedure:

Step 1: user and system information,

Command : "whoami"

```
(kalipodo_saha@Kalipodo)-[~]  
$ whoami  
kalipodo_saha
```

Step 2: Directory navigation.

Command : "pwd"

"ls"

"cd"

"cd .."

"cd ~"

"cd /"

```
(kalipodo_saha@Kalipodo)-[~]
$ pwd
/home/kalipodo_saha

(kalipodo_saha@Kalipodo)-[~]
$ ls
Burpsuite-Professional  Pictures  kali-sign-venv
Desktop                 Public    kudip
Dev                     Templates nano.26669.save
Documents               Videos   nano.5390.save
Downloads               go        sherlock
'Kuldip Saha Mugdha.txt' hi
Music                   i.txt
```

```
(kalipodo_saha@Kalipodo)-[~]
$ cd Desktop

(kalipodo_saha@Kalipodo)-[~/Desktop]
$ cd ~

(kalipodo_saha@Kalipodo)-[~]
$ cd /

(kalipodo_saha@Kalipodo)-[/]
$ cd ..
```

Step 3: File and directory management:

Command: 'mkdir Kuldip'

```
jdk-24_linu...
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ mkdir Kuldip

Kuldip
(kalipodo_saha@Kalipodo)-[~/Desktop]
```


"rm dir Kuldip"

```
(kalipodo_saha@Kalipodo) - [~/Desktop]  
$ rm -r Kuldip
```

"rm < mugdha.txt"

```
(kalipodo_saha@Kalipodo) - [~/Desktop]  
$ rm mugdha.txt  
  
(kalipodo_saha@Kalipodo) - [~/Desktop]  
$
```

"rm -r Kuldip"

```
(kalipodo_saha@Kalipodo) - [~/Desktop]  
$ rm -r Kuldip  
  
(kalipodo_saha@Kalipodo) - [~/Desktop]  
$
```

"cp mugdha.txt Dev"

```
(kalipodo_saha@Kalipodo) - [~/Desktop]  
$ cp mugdha.txt Dev  
  
(kalipodo_saha@Kalipodo) - [~/Desktop]  
$
```


"mv mugdha.txt Dev"

```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ mv mugdha.txt Dev

(kalipodo_saha@Kalipodo)-[~/Desktop]
$ |
```

"touch mugdha.txt"

```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ touch mugdha.txt

(kalipodo_saha@Kalipodo)-[~/Desktop]
$ rm mugdha
```

Step 4: Searching file:

Command: "find /dev -name mugdha.txt"

```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ find /dev -name mugdha.txt

(kalipodo_saha@Kalipodo)-[~/Desktop]
$ |
```

"locate mugdha.txt"

```
(kalipodo_saha@Kalipodo)-[~/Desktop/Dev]
$ locate mugdha.txt

(kalipodo_saha@Kalipodo)-[~/Desktop/Dev]
$ |
```

"grep 'kuldip' mugdha.txt"

```
(kalipodo_saha@Kalipodo)-[~/Desktop]  
$ grep 'kuldip' mugdha.txt  
i'm kuldip
```

Step 5: Reviewing file content:

Command: "cat mugdha.txt"

"more mugdha.txt"

"less mugdha.txt"

"head mugdha.txt"

"tail mugdha.txt"

```
(kalipodo_saha@Kalipodo)-[~/Desktop]  
$ cat mugdha.txt
```

```
hi  
i'm kuldip  
it22018
```

```
(kalipodo_saha@Kalipodo)-[~/Desktop]  
$ head mugdha.txt
```

```
hi  
i'm kuldip  
it22018
```

```
(kalipodo_saha@Kalipodo)-[~/Desktop]  
$ tail mugdha.txt
```

```
hi  
i'm kuldip  
it22018
```



```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ more mugdha.txt
hi
i'm kuldip
it22018
```

Step-6: Text editor:

Command : "pic o mugdha.txt"
" nano mugdha.txt "
" mousepad mugdha.txt "

Step-7: File permission:

Command : " Chmod 777 mugd kuldip.txt "

```
(kalipodo_saha@Kalipodo)-[~/Desktop/kuldip]
$ chmod 777 kuldip.txt

(kalipodo_saha@Kalipodo)-[~/Desktop/kuldip]
$ ls -la
total 8
drwxrwxr-x  2 kalipodo_saha kalipodo_saha 4096 Dec  2 08:40 .
drwxr-xr-x 16 kalipodo_saha kalipodo_saha 4096 Dec  2 08:40 ..
-rwxrwxrwx  1 kalipodo_saha kalipodo_saha   0 Dec  2 08:40 kuldip.txt
```

Step 8: ~~to~~ Compression :

Command : " zip kuldip.zip kuldip "

```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ zip kuldip.zip kuldip
adding: kuldip/ (stored 0%)
```


" unzip kuldip.zip "

```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ unzip kuldip.zip
Archive:  kuldip.zip
```

Step-9: Package installation:

Command: " sudo apt-get update "

```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ sudo apt-get update
[sudo] password for kalipodo_saha:
Get:1 https://dl.google.com/linux/chrome/deb stable InRelease [1825 B]
Get:2 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:4 https://dl.google.com/linux/chrome/deb stable/main amd64 Packages [1210 B]
33% [3 Packages 15.6 MB/21.0 MB 75%]
```

Step-10: SSH login

Command: " ssh kalipodo_saha@ssh.student.bth.se "

```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ ssh kalipodo_saha@ssh.student.bth.se
The authenticity of host 'ssh.student.bth.se (194.47.129.188)' can't be established.
ECDSA key fingerprint is: SHA256:MxVDvqqadEpl1+/KBz5FC8Pl9WFX+I1MfkH5IC4ZR8M
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Step-11: Shutdown:

Command: " shutdown -h now "

```
(kalipodo_saha@Kalipodo)-[~/Desktop]
$ shutdown -h now
```

Result:

Here we used the command for file and directory, navigation, permission, ~~networking~~. we got our proper result as expected. we found no vulnerability in our experiment.

Observation:

- All command execute successfully.
- File management options (create, delete, move, copy) worked as expected.
- permission changes were accurately reflected in the ls-la output.
- system monitoring commands provided real-time resource access.

Discussion:

Interpretation of Results: The commands behaved exactly as described in linux manuals. The experiment verified that linux can fully control the operating system through the terminal without relying on GUI Tools.

Comparison with Theoretical expectations:

- Theory → Linux command allows direct communication with the OS Kernel.

- Practical → All tested commands matched the theoretical behavior precisely.

Conclusion:

The experiment successfully demonstrated the use of essential Linux commands for file management, directory navigation, permissions, networking and system administration.

References:

- GNU Linux Documentation
- Linux Manual Pages.
- The linux documentation project
- Online resources.

Appendices:

Appendix A → Complete Linux command list

Appendix B → Screenshot placement guide.



MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Santosh, Tangail-1902

LAB REPOT

Department of : Information and Communication Technology

Lab Report No : 04

Lab Report On : Linux Command for networking

Course Title : Computer Network Lab

Course Code : ICT-3202

Submitted by

Name : Kuldeep Saha Majda

ID : IT22018 Session : 2021-22

Year : 1st Semester : 2nd

Dept. of ICT, MBSTU

Submitted to

Mr. ~~Mr.~~ Dr. Nazmul Islam

Associate professor

Dept. of ICT, MBSTU

Date of Performance : 2/12/25

Date of Submission : 9/12/25

Introduction:

Purpose of the experiment: The purpose of this experiment is to understand practical Linux networking concepts including IP addressing, Mac identification, routing table interpretation, creating virtual interfaces, adding new networks, and configuring with multiple network routing scenarios.

Background information: Linux provides a robust set of tools for managing networking. Commands such as `ifconfig`, `ip addr`, `ip route`, `netstat -r` and interface manipulation allow users to analyze and configure network behavior.

Hypothesis: If Linux networking commands are applied correctly, then the system should successfully display network information, create virtual interfaces, update routing tables and manage multiple networks without errors.

Objectives:

- i) Identify IP and MAC address
- ii) Understand routing table
- iii) Create and remove virtual interfaces.
- iv) Add or delete network routes.
- v) Configure a multi-network scenario and assign correct gateways.

Materials and Methods:

List of Materials:

- A Linux system
- Terminal access with sudo privileges.
- Internet.
- VMware.

Procedure:

[1] Why hosts can only use addresses 192.168.1.1 to 192.168.1.254:

In a subnet the address 192.168.1.0 and 192.168.1.255 is reserved. Hence,

- 192.168.1.0 is used as Network address and
- 192.168.1.255 is used as Broadcast address.

And for this only the other addresses are used by the hosts.

[2] IP and MAC address identification:

Using Linux network-tools, I retrieved the network configuration of my laptop.

IP address: 10.0.2.15

Mac address: 08:00:27:49:9d:dd.


```

$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f4:94:dd brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 82879sec preferred_lft 82879sec
    inet6 fd17:625c:f037:2:4dfc:97ba:b4e7:b7c3/64 scope global temporary dynamic
        valid_lft 86113sec preferred_lft 14113sec
    inet6 fd17:625c:f037:2:a00:27ff:fe4:94dd/64 scope global dynamic mngtaddr

```

③ Routing Table Basics:

Routing table commands are given below:

~~Destination:~~ ip route

sudo ip addr add 10.0.2.0/29 dev enp0s3 label enp0s3:1

sudo ip link set enp0s3:1 up

```

$ sudo apt install net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
net-tools is already the newest version (2.10-0.1ubuntu4.4).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
$ netstat -r
Kernel IP routing table

```

Destination	Gateway	Genmask	Flags	MSS Window	irtt	Iface
default	gateway	0.0.0.0	UG	0 0		0 enp0s3
10.0.2.0	0.0.0.0	255.255.255.0	U	0 0		0 enp0s3
192.168.2.0	0.0.0.0	255.255.255.0	U	0 0		0 enp0s3

④ Virtual Interfaces:

a) Creating Virtual Interfaces:

• ~~sudo~~ ip addr show enp0s3

b) Adding a route for the virtual interface:

- `Sudo ip route add 10.0.2.0/24 dev enp0s3:1`

c) Removing the route:

- `Sudo ip route del 10.0.2.0/24 dev enp0s3:1`

d) Removing the route virtual interface:

- `Sudo ip link set enp0s3:1 down.`

```
ubuntu@ubuntu: $ sudo ip addr add 10.0.2.1/8 dev enp0s3
ubuntu@ubuntu: $ sudo ip addr add 192.168.1.25/24 dev enp0s3
ubuntu@ubuntu: $ sudo ip link set enp0s3 up
ubuntu@ubuntu: $ sudo ip link set enp0s3 up
ubuntu@ubuntu: $ sudo ip route add 10.0.2.0/24 dev enp0s3
ubuntu@ubuntu: $ sudo ip route add 192.168.1.0/24 dev enp0s3
ubuntu@ubuntu: $ sudo ip route add 192.168.1.0/24 dev enp0s3
RTNETLINK answers: File exists
```

5) Adding a new network:

a) Add a new network route:

- `Sudo ip addr add 10.0.2.0/24 dev enp0s3 label enp0s3:2`
- `Sudo ip link set enp0s3:2 up`
- `Sudo ip route add 10.0.2.0/24 dev enp0s3:2`
- `ip addr show enp0s3`

b) Assign new gateway for newly added network:

- `Sudo ip route add default via 10.0.2.2 dev enp0s3:2`
- `ip route show.`

c) Verify the routing table:

`netstat -n`

6) Multi-Network scenario configuration.

- a). `sudo ip addr add 10.0.2.1 /8 dev enp0s3`
 - `sudo ip addr add 192.168.1.25/24 dev enp0s3`
 - `sudo ip link set enp0s3 up`
 - `sudo ip link set enp0s3 up`
- b). `sudo ip route add 10.0.2.0/24 192.168.1.0/24 dev enp0s3`
- c). `sudo ip route add default via 192.168.1.1 dev enp0s3`
 - `ip route show.`

```
ubuntu@ubuntu:~$ sudo ip route add default via 192.168.1.1 dev enp0s3
ubuntu@ubuntu:~$ ip route show
default via 192.168.1.1 dev enp0s3
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.0.0/8 dev enp0s3 proto kernel scope link src 10.0.2.1
10.0.2.0/24 dev enp0s3 scope link
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
192.168.1.0/24 dev enp0s3 proto kernel scope link src 192.168.1.25
192.168.2.0/24 dev enp0s3 proto kernel scope link src 192.168.2.32
ubuntu@ubuntu:~$ sudo systemctl --w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

Safety Precautions.

- Avoid deleting critical system routes.
- Don't modify network interfaces if performing on a production machine.
- Ensure proper authorization before network configuration.


```

ubuntu@ubuntu: ~$ sudo ip addr add 10.0.2.50/24 dev enp0s3 label en
p0s3:2
ubuntu@ubuntu: ~$ sudo ip link set enp0s3:2 up
ubuntu@ubuntu: ~$ sudo ip route add 10.0.2.0/24 dev enp0s3:2
ubuntu@ubuntu: ~$ ip addr show enp0s3
ubuntu@ubuntu: ~$ ip route show
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 scope link
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
ubuntu@ubuntu: ~$ netstat -r
Kernel IP routing table
Destination        Gateway            Genmask           Flags   MSS Window  irtt Iface
default            _gateway          0.0.0.0           UG        0  0          0 enp0s3
10.0.2.0           0.0.0.0           255.255.255.0     U        0  0          0 enp0s3
10.0.2.0           0.0.0.0           255.255.255.0     U        0  0          0 enp0s3
ubuntu@ubuntu: ~$ sudo ip route del 10.0.2.0/24 dev enp0s3:2
ubuntu@ubuntu: ~$ sudo ip route del default via 10.0.2.0/24 dev enp0s3:2

```

Result:

Data presentation:

- IP and MAC address.
- Routing table entries.
- Newly created virtual interface.
- Added/removed network entries.
- Multi network routing configuration.

Observation:

- It config successfully displayed system - assigned IP and MAC.
- Routing table columns matched theoretically after definitions.
- Virtual interface etho:1 appeared automatically after creating.

Discussion:

Interpretation of results:

The experiments demonstrated how Linux handles routing interfaces and network segmentation. Each command functioned as predicted, confirming Linux's reliability in network administration tasks.

Source of error:

- Incorrect netmask may cause routing failure
- Wrong gateway assignment disconnect machine
- Permission issue if sudo not used.
- Using inactive interface names.

Suggestions for Improvement:

- Use ip command instead of legacy ifconfig and route
- Automate setup with shell scripts.
- Implement firewall rules in future labs.

Further Research:

- VLAN configurations
- NAT and PAT
- DHCP and DNS configuration.
- VPN routing.

Conclusion:

Summary of finding:

The experiment successfully demonstrated Linux networking operations including viewing IP/MAC, interpreting routing tables, configuring virtual interfaces.

Reflection on Hypothesis:

The hypothesis was supported - all commands worked correctly when applied properly.

Practical implications:

These skills are essential in:

- System administration.
- Ethical hacking / cyber security
- Networking engineering
- Server management
- Cloud deployment.

References:

- Linux Networking Documentation.
- Ubuntu Man pages (man ifconfig, man route, man netstat)
- Linux Foundation tutorials.

Appendices:

Appendix A - Command list

- ifconfig
- ip address
- route add
- route del.
- netstat -rn
- ifconfig eth 0:1 up/down
- sudo usage.

Appendix B - Placed all screenshots in proper place.